

Chapter 3 - Making Decisions (Questions and Answers)

Pregunta número 1

1. Which of the following data types can be used in a switch expression? (Choose all that apply.)
 - A. enum
 - B. int
 - C. Byte
 - D. long
 - E. String
 - F. char
 - G. var
 - H. double

Answer 1: El primer punto que debemos tener en cuenta es que en Java, las estructuras switch (tanto las tradicionales como las nuevas switch expressions) solo admiten un conjunto muy específico de tipos de datos que se pueden evaluar de forma determinista y eficiente mediante tablas de saltos (jump tables) en la JVM. La primera respuesta correcta es el inciso A (enum). Recordemos que los enums representan un conjunto cerrado de valores fijos y conocidos en tiempo de compilación. Son perfectos para un switch porque el compilador puede verificar que hemos cubierto todos los casos (exhaustividad). La segunda respuesta correcta es el inciso B (int). Recordemos que `int` es el tipo numérico entero base por excelencia que soporta esta estructura. La siguiente respuesta correcta es el inciso C (Byte). Java nos permite el uso de los tipos primitivos más pequeños que un entero (byte, short, char) y sus respectivas clases wrapper (Byte, Short, Character, Integer). El compilador se encarga de hacer el unboxing de la clase Byte al primitivo byte automáticamente. La cuarta respuesta correcta es el inciso E (String), ya que se soporta desde Java 7. Detrás de escena, el switch evalúa el hashCode() del String y luego usa equals() para confirmar la coincidencia, evitando colisiones. La quinta respuesta correcta es el inciso F (char), ya que, al igual que byte y short, un char se representa internamente en Java como un valor numérico entero (de 16 bits), por lo que es totalmente compatible. Finalmente, el último inciso correcto es el inciso G (var). Aunque var no es un tipo de dato real, sino una palabra reservada para la inferencia de tipos local, una variable declarada con var sí puede usarse como selector en un switch. Esto funcionará siempre y cuando el tipo que el compilador infiera pertenezca a la lista de los permitidos (por ejemplo, si infiere que es un String o int). En resumen, un switch solo acepta int, byte, short, char y sus clases wrapper, String y enum (más variables inferidas por var que resulten en estos tipos); todo lo demás (long, double, boolean, etc.) da error de compilación.

Pregunta número 2

2. What is the output of the following code snippet? (Choose all that apply.)

```
3: int temperature = 4;
4: long humidity = -temperature + temperature * 3;
5: if (temperature >= 4)
6: if (humidity < 6) System.out.println("Too Low");
7: else System.out.println("Just Right");
8: else System.out.println("Too High");
```

- A. Too Low
- B. Just Right
- C. Too High
- D. A NullPointerException is thrown at runtime.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.

Answer 2: En este problema se evalúan dos conceptos fundamentales; estos son la precedencia de operadores y el problema del dangling else (else colgante) cuando se omiten las llaves {}. Java respeta la precedencia matemática. Primero aplica el operador unario negativo (-4), y luego la multiplicación ($4 * 3 = 12$) y finalmente la suma: $-4 + 12 = 8$; por lo tanto, humidity vale 8. La anterior evaluación matemática se realiza en las líneas 3 y 4. De ahí pasamos a la estructura de control que abarca de la línea 5 a la 8. Aquí debemos recordar que en Java, la indentación (los espacios a la izquierda) no significa nada para el compilador. La regla estricta es: un else siempre pertenece al if mas cercano que no tenga ya un else asociado, a menos que usen llaves `{}` para agrupar. Con todo esto en mente, podemos afirmar que la respuesta correcta es el inciso B. Como temperature (4) es mayor o igual a 4, entra al primer if. Como humidity (8) no es menor que 6, entra el else de la línea 7, imprimiendo "Just Right". Una nota importante que debemos recalcar es que si vemos múltiples `if` y else sin llaves, no debemos confiar en los espacios o la alineación visual que nos coloque la pregunta. Considero que un truco que se puede hacer es emparejar los else de abajo hacia arriba con el if libre más cercano. Además, debemos recordar la regla "PEMDAS" para las matemáticas: paréntesis, exponentes, multiplicación/división, adición/sustracción.

Pregunta número 3

3. Which of the following data types are permitted on the right side of a for-each expression? (Choose all that apply.)
- A. `Double[][]`
 - B. `Object`
 - C. `Map`
 - D. `List`
 - E. `String`
 - F. `char[]`
 - G. `Exception`
 - H. `Set`

Answer 3: El bucle for-each (también conocido como enhanced for loop) tiene una regla muy estricta sobre lo que puede ir a la derecha de los dos puntos (:). Solo admite dos cosas: Un arreglo (Array) de cualquier tipo de dato (primitivo o de referencia) y de cualquier dimensión. Lo segundo es un objeto que implementa la interfaz `java.lang.Iterable`; esto incluye a la mayoría de las colecciones estándar de Java. El primer inciso correcto es la letra A (`Double [][]`). Esto es un arreglo bidimensional (un arreglo de arreglos) de la clase wrapper `Double`. Como es un arreglo, es perfectamente válido en el lado derecho de un for-each. La segunda respuesta correcta es el inciso D (`List`). Aquí la clave es que la interfaz `List` hereda de `Collection`, y `Collection` hereda de `Iterable`. Por lo tanto, cualquier lista es válida. El siguiente inciso correcto es la F (`char[]`), ya que es un arreglo de tipos primitivos `char`. Al cumplir con la regla de ser un arreglo, es totalmente válido. La última respuesta correcta es el inciso H (`Set`), ya que, al igual que `List`, la interfaz `Set` hereda de `Collection`, la cual a su vez implementa `Iterable`. En resumen, en los bucles for-each del lado derecho, solo arreglos (Arrays) u objetos `Iterable`.

Pregunta número 4

4. What is the output of calling `printReptile(6)`?

```
void printReptile(int category) {  
    var type = switch(category) {  
        case 1,2 -> "Snake";  
        case 3,4 -> "Lizard";  
        case 5,6 -> "Turtle";  
        case 7,8 -> "Alligator";  
    };  
    System.out.print(type);  
}
```

- A. Snake
- B. Lizard
- C. Turtle
- D. Alligator
- E. TurtleAlligator
- F. None of the above

Answer 4: El tema clave de este problema son las switch expressions. Debemos saber que, a diferencia del switch tradicional, que es una declaración de control de flujo, una switch expression evalúa y retorna un valor, en este caso, asignando a la variable `var type`. La regla de oro para las switch expressions es que deben ser exhaustivas. Esto significa que el compilador exige que cubramos absolutamente todos los posibles valores que puede tomar la variable de entrada. En el código, la variable `category` es de tipo `int`. Un `int` puede tener miles de millones de valores posibles. Sin embargo, los `case` solo cubren los números del 1 al 8. Si llamamos a `printReptile(9)` la expresión no sabría qué valor devolver a la variable `type`. Por esta razón, el código no compila y, por lo tanto, no hay salida posible. Para que este código compilara, obligatoriamente necesitaríamos tener una rama `default`. Con lo anterior dicho, la respuesta correcta es el inciso F. Un punto importante es que cuando veamos un switch, primero tenemos que identificar si es un statement (solo ejecuta código) o una expression (retorna/asigna un valor). Si es una expresión, debemos buscar inmediatamente el `default`. Si no tiene `default`, debemos verificar si el selector es un `enum` y si se listaron todos los valores del `enum`. Si es un tipo como `int`, `String` o `enum` incompleto y no hay `default`, el código no va a compilar.

Pregunta número 5

5. What is the output of the following code snippet?

```
List<Integer> myFavoriteNumbers = new ArrayList<>();
myFavoriteNumbers.add(10);
myFavoriteNumbers.add(14);
for (var a : myFavoriteNumbers) {
    System.out.print(a + ", ");
    break;
}

for (int b : myFavoriteNumbers) {
    continue;
    System.out.print(b + ", ");
}

for (Object c : myFavoriteNumbers)
    System.out.print(c + ", ");
```

- A. It compiles and runs without issue but does not produce any output.
- B. 10, 14,
- C. 10, 10, 14,
- D. 10, 10, 14, 10, 14,
- E. Exactly one line of code does not compile.
- F. Exactly two lines of code do not compile.
- G. Three or more lines of code do not compile.
- H. The code contains an infinite loop and does not terminate.

Answer 5: En esta pregunta están evaluando los conceptos de alcance de variables/tipos en los bucles for-each y el control de flujo y el código inalcanzable (unreachable code). Empezando por el primer bucle `for`, una vez que lo analizamos, puedo decir que compila perfectamente. Usar var en un bucle for-each es totalmente válido en Java (el compilador infiere que "a" es de tipo Integer). Si el código corriera, imprimiría 10, y luego el break terminaría el bucle prematuramente. Ahora, yendo al segundo bucle for, podemos notar que aquí tenemos un problema. La instrucción `continue` hace que el control del programa salte inmediatamente a la siguiente iteración del bucle. Como este continue, no está dentro de ninguna condición (if); se ejecutará siempre de forma incondicional. Por lo tanto, es absolutamente imposible que la JVM llegue a la línea del System.out.print(b + ", "). Debemos saber que en Java, el código inalcanzable (unreachable code) no es solo una advertencia, sino un error de compilación estricto. Así que esta línea exacta no compila. Por lo tanto, la respuesta correcta es el inciso E. Un punto clave que podemos remarcar es que cualquier instrucción colocada inmediatamente después

de un `break`, `continue`, `return` o `throw` incondicional generará un error de compilación por `unreachable code`.

Pregunta número 6

6. Which statements about decision structures are true? (Choose all that apply.)
- A. A `for-each` loop can be executed on any `Collections Framework` object.
 - B. The body of a `while` loop is guaranteed to be executed at least once.
 - C. The conditional expression of a `for` loop is evaluated before the first execution of the loop body.
 - D. A `switch` expression that takes a `String` and assigns the result to a variable requires a `default` branch.
 - E. The body of a `do/while` loop is guaranteed to be executed at least once.
 - F. An `if` statement can have multiple corresponding `else` statements.

Answer 6: El tema principal son las estructuras de decisión y los bucles en Java. El primer inciso correcto es el C, ya que en un bucle `for` tradicional, la secuencia exacta del compilador es inicialización, sigue con la evaluación de la condición y concluye con la ejecución del cuerpo si la condición fue verdadera. Por lo tanto, la condición sí se evalúa antes de que el cuerpo corra por primera vez. La siguiente respuesta correcta es el inciso D. Como vimos en la pregunta 4, las `switch expressions` deben ser exhaustivas. Dado que un `String` tiene una cantidad infinita de combinaciones posibles, es imposible escribir un `case` por cada una. Por ello, el compilador nos obligará estrictamente a poner un bloque `default`. El último inciso correcto es el E. Aquí tenemos la definición y característica principal del `do/while`. A diferencia del `while` normal, primero ejecuta el cuerpo del bucle (`do`) y hasta el final verifica la condición (`while`). Esto garantiza un mínimo de una ejecución.

Pregunta número 7

7. Assuming `weather` is a well-formed nonempty array, which code snippet, when inserted independently into the blank in the following code, prints all of the elements of `weather`? (Choose all that apply.)

```
private void print(int[] weather) {  
    for(_____) {  
        System.out.println(weather[i]);  
    }  
}
```

- A. `int i=weather.length; i>0; i--`
- B. `int i=0; i<=weather.length-1; ++i`
- C. `var w : weather`
- D. `int i=weather.length-1; i>=0; i--`
- E. `int i=0, int j=3; i<weather.length; ++i`
- F. `int i=0; ++i<10 && i<weather.length;`
- G. None of the above

Answer 7: El tema principal aquí es sobre los límites de los arreglos. Partamos de que el objetivo es imprimir todos los elementos del arreglo `weather`. Si observamos el cuerpo del bucle, la instrucción es `System.out.println(weather[i])`. De esa instrucción podemos tomar dos puntos clave: Por un lado, necesitamos obligatoriamente declarar una variable llamada `i` en la definición del bucle y, por el otro lado, para que no haya errores, la variable `i` debe recorrer exactamente los índices válidos de un arreglo en Java, desde 0 hasta `weather.length - 1`. La pregunta no está especificando el orden, por lo que imprimirlos de atrás hacia adelante también es válido. Con lo anterior dicho, podemos decir que los incisos correctos son el B y la D. En la B tenemos la forma segura de recorrer un arreglo hacia adelante. Empieza en 0 y termina cuando `i` es exactamente el último índice válido (`length - 1`). El uso de preincremento (`i++`) en la sección de actualización de un bucle `for` hace exactamente lo mismo. En el caso de la D, ese bucle recorre el arreglo de atrás hacia adelante. Empieza en el último índice válido (`length - 1`) y va restando hasta llegar al 0 inclusive (`i >= 0`). Imprime todos los elementos, cumpliendo la condición de la pregunta. Los puntos clave de esta pregunta son que el índice máximo es `length - 1` y que debemos tener cuidado con la sección de inicialización del `for`, ya que podemos declarar múltiples variables separadas por comas, pero no podemos repetir el tipo; o sea, podemos colocar `int a = 1, b = 2`, pero no podemos colocar `int a = 1, int b = 2`; este último no va a compilar.

Pregunta número 8

8. What is the output of calling `printType(11)`?

```
31: void printType(Object o) {
32:     if(o instanceof Integer bat) {
33:         System.out.print("int");
34:     } else if(o instanceof Integer bat && bat < 10) {
35:         System.out.print("small int");
36:     } else if(o instanceof Long bat || bat <= 20) {
37:         System.out.print("long");
38:     } default {
39:         System.out.print("unknown");
40:     }
41: }
```

- A. `int`
- B. `small int`
- C. `long`
- D. `unknown`
- E. Nothing is printed.
- F. The code contains one line that does not compile.
- G. The code contains two lines that do not compile.
- H. None of the above

Answer 8: En esta pregunta estamos evaluando pattern matching para `instanceof`. Leyendo y analizando el código, nos encontramos con el primer error de compilación en la línea 36. En esa línea estamos evaluando la regla de alcance del pattern matching. Cuando usamos `instanceof Tipo variable`, la variable solo existe si el compilador está 100% seguro de que la condición fue verdadera. Si usamos el operador AND (`&&`), el lado derecho solo se evalúa si el lado izquierdo fue true. Por lo tanto, la variable `bat` sí existe y está dentro del alcance. Pero si usamos el operador OR (`||`), el lado derecho se evalúa únicamente si el lado izquierdo fue false. Si es false, significa que o no es un Long y la variable `bat` jamás se creó. Por lo tanto, al intentar evaluar `bat <= 20`, el compilador marca un error porque `bat` no está definida (fuera de alcance). La siguiente línea que tiene error de compilación es la

38. Aquí tenemos una trampa de sintaxis. La palabra reservada default solo se puede usar dentro de una estructura switch. En una cadena de if / else if, el bloque final que atrapa cualquier otra condición siempre debe ser un else a secas. El compilador lanzará un error de sintaxis al ver un default ahí. Con toda la información anterior, podemos asegurar que la respuesta correcta es el inciso G. Dos puntos a rescatar son que una variable creada por instanceof sí puede usarse en el lado derecho de un &&, pero nunca puede usarse en el lado derecho de un ||, y que no debemos mezclar sintaxis en las estructuras de control, ya que default es exclusivo de switch, y else es exclusivo de if.

Pregunta número 9

9. Which statements, when inserted independently into the following blank, will cause the code to print 2 at runtime? (Choose all that apply.)

```
int count = 0;
BUNNY: for(int row = 1; row <=3; row++)
    RABBIT: for(int col = 0; col <3 ; col++) {
        if((col + row) % 2 == 0)
            _____;
        count++;
    }
System.out.println(count);
```

- A. break BUNNY
- B. break RABBIT
- C. continue BUNNY
- D. continue RABBIT
- E. break
- F. continue
- G. None of the above, as the code contains a compiler error.

Answer 9: Aquí el tema que estamos evaluando es el uso de etiquetas (labels) en los bucles. El objetivo del código es que, al final de los bucles, la variable count valga exactamente 2. Para entender qué hace cada instrucción, primero tenemos que tener claro cuáles son las variables en juego. En el bucle exterior (BUNNY) row va de 1 a 3 y en el bucle interior (RABBIT) col va de 0 a 2. La condición es que, si (col + row) es un número par (es decir, el residuo de la división entre 2 es 0), se ejecuta la instrucción en blanco. Si no, se incrementa count. Los incisos correctos son B, C y E. Empezando con la iteración 1, break RABBIT (inciso B) y break sin etiqueta (inciso E) hacen exactamente lo mismo: rompen el bucle interior. Mientras que continue BUNNY (inciso C) salta al siguiente ciclo del bucle exterior. Como no hay más código en el bucle exterior después del bucle interior, los tres comandos hacen que vayamos directamente a row = 2. En la iteración 2, nuevamente, las instrucciones de los incisos B, C y E hacen que salgamos de la iteración 2 y

vayamos directo a la siguiente iteración exterior (count sigue en 1). En la iteración 3, también entramos al bucle interior y las instrucciones de los incisos B, C y E hacen que salgamos y vayamos a la siguiente iteración exterior. Como row ya llegó a su límite (3), los bucles terminan. El resultado final es que se imprime 2; por lo tanto, podemos decir que los incisos B, C y E son los correctos. Los puntos clave que debemos tener presentes son que un break o continue sin etiqueta siempre afecta el bucle más cercano (el más interno) que lo envuelve. Por otro lado, un break o continue con etiqueta afecta a cualquier bucle exterior que tenga ese nombre. Un truco de diseño es que, si el bucle interior es la última o única instrucción dentro de un bucle exterior, usar `break` bucleInterior tendrá exactamente el mismo efecto práctico que usar continue bucleExterior. Ambas acciones te llevan a la siguiente iteración del bucle grande.

Pregunta número 10

10. Given the following method, how many lines contain compilation errors? (Choose all that apply.)

```
10: private DayOfWeek getWeekDay(int day, final int thursday) {  
11:     int otherDay = day;  
12:     int Sunday = 0;  
13:     switch(otherDay) {  
14:         default:  
15:             case 1: continue;  
16:             case thursday: return DayOfWeek.THURSDAY;  
17:             case 2,10: break;
```

```
18:         case Sunday: return DayOfWeek.SUNDAY;  
19:         case DayOfWeek.MONDAY: return DayOfWeek.MONDAY;  
20:     }  
21:     return DayOfWeek.FRIDAY;  
22: }
```

- A. None, the code compiles without issue.
- B. 1
- C. 2
- D. 3
- E. 4
- F. 5
- G. 6
- H. The code compiles but may produce an error at runtime.

Answer 10: En este ejercicio están evaluando las reglas estrictas de las declaraciones switch en Java, especialmente qué constituye un valor válido para un case y cómo funciona el control de flujo. En la línea 15, debemos recordar que la instrucción continue solo se puede utilizar dentro de bucles (for, while, do/while). No tiene sentido semántico dentro de un switch puro. Si queremos terminar la ejecución de ese caso y salir del switch, debemos usar break. Por otro lado, en la línea 16

tenemos la regla de oro del switch: que los valores de un case deben ser constantes en tiempo de compilación (compile-time constants). Aunque el parámetro `thursday` está marcado como final en la firma del método, su valor real se conoce hasta el tiempo de ejecución (cuando alguien invoca el método). Por ende, el compilador lo rechaza. De ahí nos vamos a la línea 18; aquí ocurre algo similar a la línea 16. La variable `sunday` fue declarada en la línea 12 como un simple `int Sunday = 0`. Al no tener el modificador final, no se considera una constante, por lo que no puede usarse como evaluador en un case. Terminamos en la línea 19; primero tenemos que notar que en la línea 13, la instrucción es `switch(otherDay)`. Como `otherDay` es un primitivo `int`, todos los case deben ser de tipo `int` (o promovibles a `int`). En esta línea se intenta evaluar contra un valor que es probablemente un enum (`DayOfWeek`). El compilador detecta inmediatamente que los tipos son incompatibles, ya que no podemos comparar un número entero con un objeto/enum. Con todo esto visto, podemos decir que la respuesta correcta es el inciso E, que justo es la que nos indica que son 4 líneas las que tienen fallos críticos que van a impedir la compilación. Los puntos clave de aquí son: primero, recordar que los valores de un case obligatoriamente deben ser constantes en tiempo de compilación (variables marcadas como final e inicializadas con un valor literal, o valores directos de un enum). Lo segundo es que los tipos de datos deben coincidir; si nuestro switch evalúa un `String`, todos sus case deben ser `String`; si evalúa un `int`, todos deben ser `int`. Finalmente, debemos recordar los límites de navegación: `break` es para salir de bucles y de switch, y `continue` es exclusivo para saltar iteraciones en bucles.

Pregunta número 11

11. What is the output of calling `printLocation(Animal.MAMMAL)`?

```
10: class Zoo {  
11:     enum Animal {BIRD, FISH, MAMMAL}  
12:     void printLocation(Animal a) {  
13:         long type = switch(a) {  
14:             case BIRD -> 1;  
15:             case FISH -> 2;  
16:             case MAMMAL -> 3;  
17:             default -> 4;  
18:         };  
19:         System.out.print(type);  
20:     } }
```

- A. 3
- B. 4
- C. 34
- D. The code does not compile because of line 13.
- E. The code does not compile because of line 17.
- F. None of the above

Answer 11: En esta pregunta estamos combinando los conceptos de switch expressions, el uso de enum y la promoción de tipos primitivos. Empezamos con la llamada al método `printLocation(Animal.MAMMAL)`, por lo que la variable `a` dentro del método toma el valor del enum `MAMMAL`. De ahí nos vamos a la expresión: está asignando un valor a la variable `type` con la sintaxis de flecha (`->`). El caso que coincide es `case MAMMAL -> 3`. En la sintaxis de flecha no existe el fall-through, lo que significa que no necesitamos un `break`; una vez que coincide el caso, retorna ese valor y sale del switch. De ahí nos vamos a la asignación y promoción de tipos. La expresión retorna el número 3 (que por defecto es un tipo primitivo `int`). La variable que recibe este valor es un `long type`. En Java, un `int` cabe perfectamente dentro de un `long`, por lo que el compilador hace una promoción implícita (widening) sin ningún problema. Por otro lado, en el bloque `default`, dado que el switch ya evaluó todos los valores posibles del enum (`BIRD`, `FISH`, `MAMMAL`), el bloque `default` es técnicamente redundante. Sin embargo, en Java, agregar un `default` a un switch exhaustivo es completamente válido y compila sin errores. Simplemente nunca se ejecutará en este escenario. Finalmente, se imprime la variable `type`, que vale 3. Con la información presente, podemos decir que la respuesta correcta es el inciso A, ya que es el resultado directo de evaluar `case MAMMAL -> 3`. Los puntos clave son la sintaxis flecha, que significa que solo se ejecuta ese bloque de código. Adiós al fall-through, adiós a la necesidad de poner `break`. El siguiente punto es la promoción de tipos, ya que una switch expression puede devolver un tipo de dato más pequeño y asignarlo a una variable de tipo más grande (por ejemplo, devolver un `int` y guardarlo en un `long`). Finalmente, el `default` redundante. Si nuestro switch

ya cubre todos los valores posibles de un enum; poner un default no causa error de compilación.

Pregunta número 12

12. What is the result of the following code snippet?

```
3: int sing = 8, squawk = 2, notes = 0;
4: while(sing > squawk) {
5:     sing--;
6:     squawk += 2;
```

```
7:     notes += sing + squawk;
8: }
9: System.out.println(notes);
```

- A. 11
- B. 13
- C. 23
- D. 33
- E. 50
- F. The code will not compile because of line 7.

Answer 12: Considero que la clave de este ejercicio tiene que ver con trazar el código a mano; digamos que nos están evaluando nuestra atención al detalle y la capacidad para seguir el valor de las variables ciclo por ciclo sin equivocarte en el uso de las matemáticas básicas. La clave es ejecutar mentalmente el bucle while paso a paso, llevando el registro exacto de las tres variables que son sing, squawk y notes. En la primera iteración tenemos que sing baja a 7, squawk sube a 4 y notes vale 11. En la segunda iteración, sing baja a 6, squawk sube a 6 y notes después de la suma vale 23. En la iteración 3, la condición que dice si $6 > 6$, sabemos que 6 no es mayor a 6; por lo tanto, el bucle termina inmediatamente. Finalmente, en la línea 9, se imprime el valor actual de notes, que es 23. Con lo anterior dicho, tenemos que la respuesta correcta es el inciso C. Este tipo de preguntas no tiene como

puntos clave más allá de tener cuidado con los valores de las variables a la hora de ir ejecutando mentalmente el código para encontrar la respuesta correcta.

Pregunta número 13

13. What is the output of the following code snippet?

```
2: boolean keepGoing = true;
3: int result = 15, meters = 10;
4: do {
5:     meters--;
6:     if(meters==8) keepGoing = false;
7:     result -= 2;
8: } while keepGoing;
9: System.out.println(result);
```

- A. 7
- B. 9
- C. 10
- D. 11
- E. 15
- F. The code will not compile because of line 6.
- G. The code does not compile for a different reason.

Answer 13: Considero que esta pregunta es parecida a la anterior en cuanto la idea es ir trazando la lógica matemática cuando, aunque en realidad, yendo paso por paso, se encuentra que el problema es puramente sintaxis. Al seguir la lógica del código, parece un bucle do/while normal; result empieza en 15, meters en 10. Restamos a meters, evaluamos si es 8 para cambiar la bandera keepGoing, y restamos 2 a result. Si el código compilara, imprimiría 11. Pero hay una trampa en la línea 8. En Java, la condición a evaluar en un bucle while o do/while siempre debe estar estrictamente encerrada entre paréntesis. La sintaxis correcta y obligatoria es } while (keepGoing). Al omitir los paréntesis, el compilador arrojará inmediatamente un error de sintaxis en esa línea. Teniendo esa información, podemos decir que la respuesta correcta es el inciso G. Creo que puedo decir que una regla importante es no dar por sentada la estructura visual de los bucles, especialmente cuando nos sentimos tentados a empezar a hacer cálculos matemáticos de inmediato. Considero que sería importante tomarnos unos segundos para revisar la “ortografía” de las estructuras de control. Debemos recordar que las condiciones para if, while, do/while, for y switch obligatoriamente requieren paréntesis () envolviendo la evaluación booleana o la variable.

Pregunta número 14

14. Which statements about the following code snippet are correct? (Choose all that apply.)

```
for(var penguin : new int[2])
    System.out.println(penguin);
var ostrich = new Character[3];
for(var emu : ostrich)
    System.out.println(emu);
List<Integer> parrots = new ArrayList<Integer>();
for(var macaw : parrots)
    System.out.println(macaw);
```

148 **Chapter 3 • Making Decisions**

- A. The data type of penguin is Integer.
- B. The data type of penguin is int.
- C. The data type of emu is undefined.
- D. The data type of emu is Character.
- E. The data type of macaw is List.
- F. The data type of macaw is Integer.
- G. None of the above, as the code does not compile.

Answer 14: Aquí estamos viendo los temas de la promoción de tipos primitivos. Hay que ir línea por línea para entender lo que está evaluando la JVM. Empezando por las líneas 12 y 13, tenemos que recordar que `double iguala = 0` es internamente `0.0`. De ahí saltamos a la línea 14, en donde estamos comparando un `double` con un `int`. Debemos tener presente que en Java, cuando aplicamos un operador de igualdad `==` entre dos tipos numéricos distintos, el compilador realiza una promoción implícita al tipo más amplio antes de compararse. El `int` se promueve temporalmente a `double`. Entonces, la comparación es `0.0 == 1.0`, lo cual es `false`. La variable `frog` vale `false`. En la línea 15 tenemos un operador ternario anidado sin paréntesis. La estructura de un ternario es `condición ? valor_si_true : valor_si_false`. Cuando no hay paréntesis, el compilador de Java asocia el segundo signo de interrogación con

los dos primeros dos puntos. Como vimos anteriormente, frog es false; por lo tanto, la JVM ignora por completo toda la expresión del medio y salta directamente al bloque falso principal, el cual es el último 12. La variable turtle toma el valor de 12. En la línea 16 se imprime el valor de turtle, que es 12. Con todo lo anterior dicho, podemos afirmar que la respuesta correcta es el inciso B, ya que es el resultado que asigna el ternario porque la primera condición (frog) es falsa. Los puntos importantes que podemos rescatar son: En la comparación de primitivos, podemos usar == entre cualquier tipo numérico primitivo (byte, short, char, int, long, float, double). Java siempre promoverá al más pequeño al más grande antes de comparar. Lo segundo es que cuando veamos el patrón A ? B ? C : D : E, el compilador siempre lo agrupará como A ? (B ? C : D) : E. Hay que evaluar siempre la primera condición (A) antes de enredarnos resolviendo lo de adentro; si la primera es falsa, saltaremos directamente a la respuesta final (E) y nos ahorraremos mucho tiempo.

Pregunta número 15

15. What is the result of the following code snippet?

```
final char a = 'A', e = 'E';
char grade = 'B';
switch (grade) {
    default:
    case a:
    case 'B': 'C': System.out.print("great ");
    case 'D': System.out.print("good "); break;
    case e:
    case 'F': System.out.print("not good ");
}
```

- A. great
- B. great good
- C. good
- D. not good
- E. The code does not compile because the data type of one or more case statements does not match the data type of the switch variable.
- F. None of the above

Answer 15: Analicemos línea por línea. Empecemos con las líneas 1 y 2; aquí todo está perfecto, los valores de un case deben ser constantes en tiempo de compilación. Como a y e están marcadas como final, se les asigna un valor literal inmediatamente; son 100% válidas para usarse dentro del switch. Hasta aquí todo bien; el problema de sintaxis lo encontramos en la línea 6. Recordemos que en Java tradicional, si queremos que múltiples casos ejecuten el mismo bloque de código (aprovechando el fall-through), debemos repetir la palabra reservada case.

Entonces, al omitir la palabra ``case`` antes de `'C'`, el compilador de Java intenta interpretar `'C'` como una etiqueta (label). Sin embargo, las etiquetas en Java deben ser identificadores válidos (como `miEtiqueta` o `bucleExterior`). Un literal de carácter como `'C'` no es un identificador válido. Por lo tanto, el compilador se rinde y arroja un error de sintaxis. Con todo lo anterior dicho, podemos afirmar que la respuesta correcta es el inciso F. El punto clave es: Agrupación de casos. Si usamos la sintaxis tradicional de dos puntos, jamás podemos poner dos valores juntos sin la palabra ``case`` en medio. Como nota, podemos mencionar que en Java 14+ si queremos agruparlos sin repetir la palabra `case`, debemos separarlos con una coma (`case 'B', 'C':`).

Pregunta número 16

16. Given the following array, which code snippets print the elements in reverse order from how they are declared? (Choose all that apply.)

```
char[] wolf = {'W', 'e', 'b', 'b', 'y'};
```

A.

```
int q = wolf.length;
for( ; ; ) {
    System.out.print(wolf[--q]);
    if(q==0) break;
}
```

B.

```
for(int m=wolf.length-1; m>=0; --m)
    System.out.print(wolf[m]);
```

C.

```
for(int z=0; z<wolf.length; z++)
    System.out.print(wolf[wolf.length-z]);
```

D.

```
int x = wolf.length-1;
for(int j=0; x>=0 && j==0; x--)
    System.out.print(wolf[x]);
```

E.

```
final int r = wolf.length;
for(int w = r-1; r>-1; w = r-1)
    System.out.print(wolf[w]);
```

F.

```
for(int i=wolf.length; i>0; --i)
    System.out.print(wolf[i]);
```

G. None of the above

Answer 16: Aquí tenemos el tema o, mejor dicho, la trampa de los índices fuera de límite (off-by-one errors) en los arreglos. El objetivo es imprimir los elementos del arreglo `wolf` en orden inverso. El arreglo tiene 5 elementos ('W', 'e', 'b', 'b', 'y'), por lo que su longitud (`wolf.length`) es 5. La regla de oro de los arreglos en Java es que los índices válidos van desde 0 hasta `length-1`. Para imprimir en reversa, necesitamos acceder estrictamente a la secuencia de índices: 4, 3, 2, 1, 0. Cualquier intento de acceder al índice 5 lanzará inmediatamente una excepción en tiempo de ejecución.

Teniendo presentes los datos anteriores, podemos decir que el primer inciso correcto es el A, ya que utiliza un bucle infinito `for ( ; ; )` que es sintácticamente válido. La variable `q` empieza en 5. En la instrucción `System.out.print(wolf[--q])`, el operador `--q` (predecremento) le resta 1 a la variable antes de usarla. Por lo tanto, en la primera iteración evalúa `wolf[4]`. Sigue bajando hasta que imprime `wolf[0]`, momento en el cual `q` es 0 y el `if(q==0) break` termina el bucle de forma segura. La segunda respuesta correcta es el inciso B, ya que es la forma estándar y más limpia de recorrer un arreglo en reversa. Empieza exactamente en `wolf.length-1` (índice 4), la condición llega hasta `>= 0` inclusive, y se decrementa en cada paso. Finalmente, la última respuesta correcta es el inciso D; este código está escrito de una forma muy confusa, pero funciona correctamente. Primero tenemos que `x` empieza en 4. El bucle `for` inicializa `j = 0`. La condición es `x >= 0 && j == 0`. Como `j` nunca cambia, esa parte siempre es verdadera, así que la condición real que controla el bucle es `x >= 0`. Entonces, `x` se decrementa (`x--`) en cada paso. Imprimirá los índices 4, 3, 2, 1, 0. Los puntos que podemos rescatar de aquí son: Los límites de los arreglos, ya que debemos tener en cuenta que el intentar acceder usando `arreglo[arreglo.length]` es incorrecto, ya que siempre debemos usar `length-1`. El otro punto es el preincremento vs. postincremento. Si vemos un arreglo `--i`, el valor de `i` baja primero y luego se accede al índice. Si vemos arreglo `[i--]`, primero se accede al índice original y después se baja el valor. Esta diferencia es vital en bucles poco ortodoxos como en el que vemos en el inciso A.

Pregunta número 17

17. What distinct numbers are printed when the following method is executed? (Choose all that apply.)

```
private void countAttendees() {  
    int participants = 4, animals = 2, performers = -1;  
    while((participants = participants+1) < 10) {}  
    do {} while (animals++ <= 1);  
    for( ; performers<2; performers+=2) {}  
  
    System.out.println(participants);  
    System.out.println(animals);  
    System.out.println(performers);  
}
```

- A. 6
- B. 3
- C. 4
- D. 5
- E. 10
- F. 9
- G. The code does not compile.
- H. None of the above

Answer 17: Aquí están poniendo a prueba el trazado de variables (desk check) que combina tres distintos bucles y pone a prueba nuestra atención al detalle con los operadores de incremento. Lo primero es calcular independientemente el valor final de cada variable para poder elegir las opciones correctas. En el bucle while (participants) tenemos inicialmente que participants = 4; de ahí tenemos la condición y algo interesante es que primero tenemos la asignación y luego evalúa la condición. De ahí tenemos que sube a 5 y, como tenemos que (5 < 10, true), sube a 6, 7, 8, 9. Sube a 10 y se evalúa 10 < 10, lo cual es falso. Aquí el bucle termina, lo que nos deja que la variable participants queda en 10. De ahí nos vamos al bucle do/while (animals); aquí iniciamos con animals = 2. Entramos al bloque do {} que realmente no hace nada. Llegamos a la condición `while (animals++ <= 1)`; aquí hay un postincremento, primero evalúa el valor actual que nos dice 2 <= 1, esto es false. Inmediatamente después de evaluar, aplica el incremento y con esto animals sube a 3. Como la condición fue falsa, el bucle termina. Por lo tanto, el valor final de animals es 3. Vamos con el bucle for (performers). Aquí iniciamos con performers = -1. La iteración 1 nos dice -1 < 2, esto es true. Se ejecuta el bloque vacío, se actualiza sumando 2 (performers += 2), por lo que ahora vale 1. En la iteración 2, en donde 1 < 2 es true. Se ejecuta el segundo bloque, se actualiza sumando 2, por lo que ahora vale 3. La iteración 3 dice que 3 < 2 es false. Con esto el bucle termina y tenemos que el valor final de performers es 3. Las líneas de impresión arrojarán 10, 3 y 3. La pregunta nos pide exactamente cuáles son los números distintos (distinct

numbers) que se imprimen; por lo tanto, las respuestas correctas son los incisos B y E.

Pregunta número 18

18. Which statements about pattern matching and flow scoping are correct? (Choose all that apply.)
- A. Pattern matching with an `if` statement is implemented using the `instance` operator.
 - B. Pattern matching with an `if` statement is implemented using the `instanceon` operator.
 - C. Pattern matching with an `if` statement is implemented using the `instanceof` operator.
 - D. The pattern variable cannot be accessed after the `if` statement in which it is declared.
 - E. Flow scoping means a pattern variable is only accessible if the compiler can discern its type.
 - F. Pattern matching can be used to declare a variable with an `else` statement.

Answer 18: Aquí tenemos los pattern matching y su regla de alcance llamada Flow Scoping. En Java 17, Oracle mejoró la forma en la que verificamos tipos y hacemos casting introduciendo el Pattern Matching. La idea principal es que, si verificas que un objeto es de un tipo, automáticamente podemos crear una variable de ese tipo en la misma línea, sin necesidad de hacer el casting explícito. La primera respuesta correcta es el inciso C, en donde tenemos el operador correcto e histórico que se utiliza y se mejoró para el pattern matching, que es `instanceof`. El otro inciso correcto es el E; aquí podemos tener en cuenta la definición de Flow Scoping (alcance por flujo) que significa que la variable declarada con pattern matching solo estará dentro del alcance en aquellas partes del código donde el compilador pueda garantizar al 100% que la condición fue verdadera y, por ende, que el tipo de la variable es seguro. El punto clave aquí es que, a diferencia de las variables normales en Java (que viven delimitadas por las llaves `{}` de su bloque), las variables de patrón viven solo donde la lógica garantice que la condición fue verdadera.

Pregunta número 19

19. What is the output of the following code snippet?

```
2: double iguana = 0;  
3: do {  
4:     int snake = 1;  
5:     System.out.print(snake++ + " ");  
6:     iguana--;  
7: } while (snake <= 5);  
8: System.out.println(iguana);
```

- A. 1 2 3 4 -4.0
- B. 1 2 3 4 -5.0
- C. 1 2 3 4 5 -4.0
- D. 0 1 2 3 4 5 -5.0
- E. The code does not compile.
- F. The code compiles but produces an infinite loop at runtime.
- G. None of the above

Answer 19: El tema principal es el alcance de las variables (scope) en las estructuras de control. La clave está en dónde nace y dónde muere la variable snake. Empezando por la línea 4, aquí la variable se declara e inicializa dentro de las llaves { } del bloque do. En Java, el ciclo de vida (scope) de cualquier variable local termina exactamente donde se cierra la llave del bloque en el que fue declarada. En la línea 7, la evaluación del while ocurre fuera del bloque do. Cuando el compilador intenta leer esta línea, la variable snake literalmente ya no existe porque su alcance terminó al cerrarse la llave }. El resultado es que el compilador detiene el proceso y lanza un error de tipo. Por lo tanto, la respuesta correcta es el inciso E, ya que, como detectamos, el error de alcance en la línea 7 impide que el programa siquiera pueda correr. El punto clave es que en cualquier bucle do/while, toda variable que vayamos a utilizar dentro de los paréntesis del while tiene que ser declarada antes de abrir las llaves del do. Si la declaramos adentro, tendremos un error de compilación inmediato por estar fuera de alcance (out of scope).

Pregunta número 20

20. Which statements, when inserted into the following blanks, allow the code to compile and run without entering an infinite loop? (Choose all that apply.)

```
4: int height = 1;
5: L1: while(height++ <10) {
6:     long humidity = 12;
7:     L2: do {
8:         if(humidity-- % 12 == 0) _____;
9:         int temperature = 30;
10:        L3: for( ; ; ) {
11:            temperature++;
12:            if(temperature>50) _____;
13:        }
14:    } while (humidity > 4);
15: }
```

- A. break L2 on line 8; continue L2 on line 12
- B. continue on line 8; continue on line 12
- C. break L3 on line 8; break L1 on line 12
- D. continue L2 on line 8; continue L3 on line 12
- E. continue L2 on line 8; continue L2 on line 12
- F. None of the above, as the code contains a compiler error

Answer 20: La clave aquí está en nuestra capacidad para rastrear variables a través de múltiples bucles anidados y el uso correcto de las etiquetas (labels). En L1 (while) que está controlado por height, que va subiendo hasta llegar a 10. Terminará naturalmente. L2 (do/while) controlado por humidity, inicia en 12 en cada ciclo de L1. La línea 8 evalúa (humidity-- % 12 == 0). Como inicia en 12, la primera vez 12 % 12 == 0 es verdadero (luego baja a 11). L3 (for). Es un for (; ;). Si el código entra a este bucle y no hay un break o un continue que apunte hacia afuera, se quedará atrapado en un bucle infinito. La primera respuesta correcta es el inciso A, ya que en la primera vuelta, humidity es 12. La condición de la línea 8 es verdadera, por lo que ejecuta break L2. Esto rompe el bloque do/while completo y pasa a la siguiente iteración de L1. El código nunca llega a entrar al bucle infinito L3. El código de la línea 12 es sintácticamente válido (compila) aunque en tiempo de ejecución nunca

se alcance. Termina con éxito. El otro inciso correcto es el E, ya que en la línea 8 humidity es 12, entra al if y ejecuta continue L2. Esto salta a la evaluación final del do/while, que es verdadera. En la siguiente iteración L2, humidity es 11, lo que hace que (11 % 12 == 0) sea falso y con esto humidity baja a 10. Pasando a la línea 12, la temperatura sube hasta 51 y ejecuta continue L2. Esto interrumpe el bucle infinito y salta otra vez a la evaluación final del do/while (10 > 4) que es verdadero. Este proceso se repite, bajando la humedad en cada vuelta. Eventualmente, humidity ya no será mayor a 4, rompiendo L2 de forma segura y pasando al siguiente ciclo de L1. Todo funciona sin bucles infinitos. La primera regla importante tiene que ver con el alcance de las etiquetas. Solo podemos usar break etiqueta o continue etiqueta si la instrucción está físicamente adentro del bloque de código de esa etiqueta. Si estamos afuera, el código no compila. Otro punto importante es el continue en bucles infinitos. Si tenemos un for (; ;) o un while(true), un continue sin etiqueta (o con la etiqueta de ese mismo bucle) no nos salva del infinito, solo reinicia la iteración. Necesitamos un break, o un continue que apunte a un bucle exterior.

Pregunta número 21

- 21.** A minimum of how many lines need to be corrected before the following method will compile?

```
21: void findZookeeper(Long id) {  
22:     System.out.print(switch(id) {  
23:         case 10 -> {"Jane"}  
24:         case 20 -> {yield "Lisa"}};  
25:         case 30 -> "Kelly";  
26:         case 30 -> "Sarah";  
27:         default -> "Unassigned";  
28:     });  
29: }
```

- A. Zero
- B. One
- C. Two
- D. Three
- E. Four
- F. Five

Answer 21: En este ejercicio tenemos que tener en cuenta tres reglas completamente distintas de las switch expressions en un solo bloque de código. Tenemos tres errores en tres líneas diferentes, empezando por la línea 21; aquí el punto es el tipo de dato del switch, ya que, como vimos anteriormente en otros ejercicios, las estructuras switch nunca han soportado el tipo long ni su clase

wrapper Long. El parámetro id es de tipo Long, por lo que la instrucción switch(id) falla inmediatamente. Para arreglar esto, necesitamos corregir una línea, ya sea cambiando la firma del método en la línea 21 a Integer id, o haciendo un cast en la línea 22 como switch((int)(long)id). De ahí nos vamos a la línea 23; en esta línea la clave está en la sintaxis del retorno de bloques. Tenemos que recordar que en una switch expression, si usamos la sintaxis de flecha y abrimos llaves {} para crear un bloque de código, estamos obligados a usar la palabra reservada yield para retornar el valor, seguido de un punto y coma. La línea 23 dice case 10 -> {"Jane"}, lo cual es sintaxis inválida. Para corregirlo, tenemos dos opciones: la primera es cambiar esa línea por case 10 -> "Jane" o también podemos cambiar la línea por case 10 -> {yield "Jane"}. De ahí nos vamos a la línea 26. En las líneas 25 y 26 evaluamos exactamente el mismo valor: case 30. El compilador de Java exige que todos los valores de los casos en un switch sean únicos. Al detectar el segundo case 30, marcará un error de etiqueta duplicada (duplicate case label). Necesitamos eliminar o cambiar el número en la línea 26. Por lo tanto, la respuesta correcta es el inciso D. Los puntos que me gustaría resaltar de este ejercicio son: Hay tipos incompatibles, lo que hace que necesitemos verificar la variable que va al switch, recordando que long, double y boolean están prohibidos. De ahí nos vamos a la sintaxis de flechas y bloques. Si hay llaves {}, debemos buscar el yield y su punto y coma. Si no hay llaves, solo debe haber una expresión o valor simple. Finalmente, es importante revisar que en ningún case se evalúe el mismo literal.

Pregunta número 22

22. What is the output of the following code snippet? (Choose all that apply.)

```
2: var tailFeathers = 3;
3: final var one = 1;
4: switch (tailFeathers) {
5:     case one: System.out.print(3 + " ");
6:     default: case 3: System.out.print(5 + " ");
7: }
8: while (tailFeathers > 1) {
9:     System.out.print(--tailFeathers + " "); }
```

- A. 3
- B. 5 1
- C. 5 2
- D. 3 5 1
- E. 5 2 1
- F. The code will not compile because of lines 3–5.
- G. The code will not compile because of line 6.

Answer 22: En este ejercicio se combina la inferencia de tipos con var junto con las reglas de las constantes en un switch. Hay que ir línea por línea para entender en profundidad qué está sucediendo. Empezando por la línea 2, la variable se inicializa en 3 (el compilador infiere que es de tipo int). De ahí nos vamos a la línea 3; aquí tenemos un concepto clave. Al usar `var` con un número literal (1), Java infiere el tipo int. Al agregarle la palabra reservada final, se convierte en una constante en tiempo de compilación (compile-time constant). Esto significa que es totalmente válida para usarse dentro de un case. Pasemos a la estructura switch que tenemos entre las líneas 4 y 7. Entramos al switch con el valor de tailFeathers, que es 3. El case one (que equivale al case 1) no coincide. Llegamos a la línea 6 donde tenemos default: case 3. En Java, es totalmente legal apilar múltiples etiquetas (labels) de un switch en la misma línea. Como el valor es 3, entra por el case 3 y ejecuta el print, por lo que se imprime 5. Como es la última instrucción del bloque switch, simplemente sale de él. No hace falta un break. En las líneas 8 y 9, donde tenemos el bucle while, debemos comenzar con la iteración 1; aquí evaluamos la condición, que es verdadera, por lo que entramos al bucle. Ejecuta el print. El operador – está a la izquierda (predecremento), así que primero resta 1 a la variable (bajándola a 2) y luego la usa para imprimir; se imprime 2. En la iteración 2, se evalúa la condición con el nuevo valor; sigue siendo verdadera la condición, por lo que volvemos a entrar. Ejecuta el predecremento, bajando a 1, y luego la imprime; lo que se imprime es 1. En la iteración 3, se evalúa la condición; ahora es falsa, por lo que el bucle termina. Juntando todas las impresiones, la salida exacta en consola es 5 2 1. Por lo tanto, el inciso correcto es el E, ya que es exactamente el resultado. Los puntos clave son: Un final var inicializado con un literal es una constante válida en tiempo de compilación y puede usarse como valor en un case. Otro punto clave es que en los switch tradicionales podemos poner tantas etiquetas juntas como queramos. Finalmente, debemos recordar que siempre debemos leer de izquierda a derecha; si el predecremento (–) está antes de la variable, la resta ocurre antes de que la línea de código haga cualquier otra cosa con ella.

Pregunta número 23

23. What is the output of the following code snippet?

```
15: int penguin = 50, turtle = 75;  
16: boolean older = penguin >= turtle;  
17: if (older = true) System.out.println("Success");  
18: else System.out.println("Failure");  
19: else if(penguin != 50) System.out.println("Other");
```

- A. Success
- B. Failure
- C. Other
- D. The code will not compile because of line 17.
- E. The code compiles but throws an exception at runtime.
- F. None of the above

Answer 23: En este ejercicio tenemos el operador de asignación dentro de un if y el orden de la estructura if-else; esos son los puntos clave. En las líneas 15 y 16 tenemos que penguin vale 50 y turtle vale 75. La condición `penguin >= turtle` (`50 >= 75`) es falsa, así que la variable booleana `older` se inicializa como `false`. Hasta aquí, todo bien. De ahí, yendo a la línea 17 (`if (older = true)`): Aquí nos debemos fijar en que hay un solo signo de igual (`=`), que es el operador de asignación, no el de comparación (`==`). En Java, el resultado de una asignación es el valor que se asignó. Así que `older = true` le asigna `true` a la variable y luego evalúa toda la expresión como `true`. Como la estructura `if` en Java requiere un valor booleano, esta línea compila perfectamente. Si el código corriera, entraría a este bloque. En la línea 18 (`else`) está el bloque que se ejecutaría si la condición del `if` fuera falsa. Sintácticamente es correcto y cierra la cadena del `if` de la línea 17. De ahí pasamos a la línea 19 (`else if...`): Aquí es donde tenemos el error de compilación. La regla fundamental de las estructuras de decisión en Java es que el bloque `else` (sin `if`) debe ser obligatoriamente el último eslabón de la cadena. No podemos poner un `else if` después de un `else`. El compilador de Java leerá esto y nos lanzará un error indicando que ese `else if` no tiene un `if` correspondiente que lo inicie. Con todo lo anterior dicho, tenemos que la respuesta correcta es el inciso F. Los dos puntos clave a rescatar son que debemos tener cuidado con la asignación vs. igualdad, sobre todo cuando veamos un solo `=` dentro de un `if` o un `while`. Si la variable es booleana, sí compila. Si es cualquier otro tipo de dato, no compila. El otro punto es que el orden es sagrado. Debemos respetar el orden estructural inquebrantable: primero `if`, luego cero o más `else if`, y al final, opcionalmente, un solo `else`. Jamás podemos poner nada después de un `else`.

Pregunta número 24

24. Which of the following are possible data types for `friends` that would allow the code to compile? (Choose all that apply.)

```
for(var friend in friends) {  
    System.out.println(friend);  
}
```

- A. Set
- B. Map
- C. String
- D. `int[]`
- E. Collection
- F. `StringBuilder`
- G. None of the above

Answer 24: Si leemos el código rápido por encima, nos puede pasar que lo corregiremos automáticamente para que parezca un bucle for-each válido. Sin embargo, si miramos con calma y lupa la declaración del bucle: `for(var friend in friends)`, tenemos que en Java, el bucle for-each (o enhanced for loop) siempre y de forma estricta utiliza los dos puntos (`:`) para separar la variable de la colección o arreglo que se está iterando. El uso de la palabra ``in`` es propio de lenguajes como Python o JavaScript, pero en Java simplemente no existe en este contexto. Esto provocará un error de compilación inmediato por sintaxis inválida. Debido a esto, no importa en absoluto qué tipo de dato le asignemos a la variable `friends`; el código jamás logrará compilar. Por ende, el inciso correcto es el G. Creo que aquí un punto clave es tener cuidado con las palabras clave como `in`, `function`, `def` o `instanceof`, que pertenecen a otros lenguajes y que nos pueden engañar a primera vista. Por otro lado, la sintaxis sagrada del for-each de Java siempre es `for (Tipo variable : coleccion_o_arreglo)`. No debemos asumir que un código compila hasta revisar la “ortografía” de sus operadores clave.

Pregunta número 25

25. What is the output of the following code snippet?

```
6: String instrument = "violin";
7: final String CELLO = "cello";
8: String viola = "viola";
9: int p = -1;
10: switch(instrument) {
11:     case "bass" : break;
12:     case CELLO : p++;
13:     default: p++;
14:     case "VIOLIN": p++;
15:     case "viola" : ++p; break;
16: }
17: System.out.print(p);
```

Review Questions

- A. -1
- B. 0
- C. 1
- D. 2
- E. 3
- F. The code does not compile.

Answer 25: Aquí estamos combinando el fall-through (caída libre) de los switch, la sensibilidad a mayúsculas y minúsculas y las variables distractoras. Hay que ir línea por línea para saber qué está sucediendo exactamente. Empezamos con las variables iniciales en las líneas 6-9: tenemos instrument = "violin" (todo en minúsculas), CELLO = "cello" (constante) y viola = "viola", que es una variable normal que funciona más como un distractor. De ahí nos vamos a la evaluación del switch en las líneas 10-15: en esta parte el switch busca una coincidencia exacta para el texto "violin". El case "bass" es falso. El case CELLO ("cello") es falso. El case "VIOLIN" también es falso, ya que en Java, los String son sensibles a mayúsculas y minúsculas (case-sensitive), por lo tanto, "violin" no es igual a

"VIOLIN". El case "viola" es falso. Debemos aclarar que usa el literal "viola", no la variable de la línea 8, por lo que va a compilar perfectamente. De ahí nos vamos al bloque default y el fall-through; como no hubo ninguna coincidencia exacta, el flujo de ejecución salta directamente al bloque default de la línea 13. Ejecuta p++. La variable p sube de -1 a 0. Aquí tenemos una trampa de fall-through. El bloque default no tiene un break. Por lo tanto, la ejecución "cae" al siguiente caso sin evaluar si coincide o no. Entonces pasamos al case "VIOLIN" y ejecuta p++. La variable p sube de 0 a 1. Tampoco hay break aquí. De ahí caemos al case "viola" y ejecuta ++p. La variable p sube de 1 a 2. Aquí sí encuentra un break, por lo que sale del switch. De ahí tenemos la impresión de la línea 17, donde se imprime el valor final de p, que es 2. Con lo anterior dicho, podemos decir que el inciso correcto es el D. Los puntos clave son la sensibilidad de mayúsculas; tenemos que recordar que las comparaciones de String en los switch son exactas, ya que "A" no es igual a "a". También debemos tener cuidado con los default flotantes. El bloque default no tiene que estar al final del switch. Si se activa y no tiene un break, ejecutará todos los casos que estén por debajo de él hasta encontrar un break o el fin de la estructura.

Pregunta número 26

26. What is the output of the following code snippet? (Choose all that apply.)

```
9:  int w = 0, r = 1;
10:  String name = "";
11:  while(w < 2) {
12:      name += "A";
13:      do {
14:          name += "B";
15:          if(name.length()>0) name += "C";
16:          else break;
17:      } while (r <=1);
18:      r++; w++; }
19:  System.out.println(name);
```

- A. ABC
- B. ABCABC
- C. ABCABCABC
- D. Line 15 contains a compilation error.
- E. Line 18 contains a compilation error.
- F. The code compiles but never terminates at runtime.
- G. The code compiles but throws a `NullPointerException` at runtime.

Answer 26: Siento que la clave de este ejercicio es ver el alcance de las llaves {} y las condiciones de los bucles anidados. Hay que trazar el estado de las variables desde el principio para ver qué sucede. El estado inicial lo tenemos en las líneas 9-10: `w = 0, r = 1, name = ""`. La entrada al bucle `while` exterior (línea 11) donde la condición es `0 < 2`, es verdadera, por lo que sí entramos al bucle. En la línea 12 tenemos `name += "A"`. Ahora `name` vale "A". De ahí nos vamos a la entrada al bucle `do/while` interior en las líneas 13-17. En la iteración 1 tenemos la línea 14: `name += "B"`. Ahora `name` vale "AB". En la línea 15 se pregunta si `name.length() > 0`; esta condición es `true`, ya que la longitud es 2. Entonces `name += "C"`, por lo que ahora `name` vale "ABC". En la línea 17 tenemos la condición `r <= 1`; como `r` vale 1, la condición es verdadera (`1 <= 1`). El bucle interior vuelve a empezar. En la iteración 2 se vuelve a agregar "B" y "C", así que `name` se vuelve "ABCBC". En la línea 17 tenemos la condición `r <= 1`, que sigue siendo verdadera. La línea 18 (`r++; w++;` }) está fuera del bucle `do/while`. Como la variable `r` nunca se incrementa dentro del bloque del `do/while`, su valor siempre será 1. El programa se quedará atascado para siempre evaluando `1 <= 1` y agregando "BC" a la cadena de texto hasta colapsar. Por lo que podemos afirmar que la respuesta correcta es el inciso F. El punto clave es que cuando veamos bucles anidados, antes de empezar a trazar valores matemáticos, tenemos que preguntarnos si las variables que controlan la condición del bucle se están actualizando realmente dentro de ese mismo bucle. Si la respuesta es no, tenemos un bucle infinito garantizado. Debemos fiarnos de las llaves {}, no de la indentación del código.

Pregunta número 27

27. What is printed by the following code snippet?

```
23: byte amphibian = 1;
24: String name = "Frog";
25: String color = switch(amphibian) {
26:     case 1 -> { yield "Red"; }
27:     case 2 -> { if(name.equals("Frog")) yield "Green"; }
28:     case 3 -> { yield "Purple"; }
29:     default -> throw new RuntimeException();
30: };
31: System.out.print(color);
```

154 Chapter 3 • Making Decisions

- A. Red
- B. Green
- C. Purple
- D. RedPurple
- E. An exception is thrown at runtime.
- F. The code does not compile.

Answer 27: Aquí vuelve a surgir como tema principal las reglas de las switch expressions que devuelven un valor. Hay que revisar la estructura de esta expresión switch línea por línea para entender qué está sucediendo. En las líneas 23 y 24 tenemos que amphibian se declara como un byte con valor 1 y name como un String con valor "Frog". Todo perfecto hasta este momento. Los tipos byte son completamente válidos para evaluar en un switch. En la línea 25 se inicia la expresión switch. Se espera que esta estructura devuelva un valor String para asignarlo a la variable color. De ahí, en la línea 26 (case 1 -> { yield "Red"; }): tenemos una sintaxis impecable. En un bloque con llaves {}, usamos `yield` para retornar el valor. En la línea 27 (case 2 -> { if(name.equals("Frog")) yield "Green"; }):

tenemos finalmente el error, ya que en una expresión switch que devuelve un valor, el compilador es extremadamente estricto: cada bloque debe garantizar que se devuelva un valor bajo cualquier circunstancia (o que se lance una excepción). En esta línea, ¿qué pasa si name no es igual a "Frog"? El bloque `if` se saltaría y el código llegaría al final de la llave `}` sin haber ejecutado ningún `yield`. Como el compilador detecta que existe una ruta lógica que no devuelve nada, detiene la compilación y arroja un error exigiendo un resultado. En las líneas 28 y 29, el case 3 y el default están correctamente estructurados (el default lanza una excepción, lo cual es una salida válida para una rama del switch). Para que este código compilara, la línea 27 tendría que cubrir todos los escenarios posibles. Con todo lo anterior presente, tenemos que el inciso correcto es el F. Los puntos clave son saber que en las switch expressions, la exhaustividad tiene dos reglas de oro: Por un lado, debemos cubrir todos los valores posibles de entrada (normalmente se logra con un default). Y por el otro lado, cada caso debe devolver un valor en el 100% de sus rutas internas. No podemos dejar un if sin su else dentro de un bloque case si de ahí depende el yield.

Pregunta número 28

28. What is the output of calling `getFish("goldie")`?

```
40: void getFish(Object fish) {  
41:     if (!(fish instanceof String guppy))  
42:         System.out.print("Eat!");  
43:     else if (!(fish instanceof String guppy)) {  
44:         throw new RuntimeException();  
45:     }  
46:     System.out.print("Swim!");  
47: }
```

- A. Eat!
- B. Swim!
- C. Eat! followed by an exception.
- D. Eat!Swim!
- E. An exception is printed.
- F. None of the above

Answer 28: Nos volvemos a encontrar las reglas de flow scoping en el pattern matching que vimos algunas preguntas atrás. Para entender qué sucede con el código, tenemos que seguir paso a paso la vida de la variable de patrón guppy y dónde el compilador considera que está dentro del alcance (in scope). Empezamos con la llamada al método: Se invoca `getFish("goldie")`, por lo que el parámetro fish entra como un String válido. En la línea 41 (`if (!(fish instanceof String guppy))`): el

código intenta hacer un pattern matching. La variable guppy solo existirá en los lugares donde el compilador pueda garantizar 100% que fish es un String. Si entra al bloque if (Línea 42), significa que la evaluación fue verdadera (o sea, fish no era un String). Por lo tanto, guppy no existe en la línea 42. Si se va por el else, significa que la evaluación fue falsa. Esto solo ocurre si fish sí era un String. Por lo tanto, en la rama else, el compilador está absolutamente seguro de que fish es un String, así que la variable guppy sí está disponible en todo el bloque else. En la línea 43 (else if (!(fish instanceof String guppy))): ocurre el error de compilación. Como acabamos de deducir, la variable guppy (creada en la línea 41) ya está viva y dentro del alcance en esta rama else. Al poner instanceof String guppy de nuevo, le estamos diciendo a Java que declare una nueva variable llamada guppy. En Java no podemos declarar dos variables locales con el mismo nombre en el mismo alcance. El compilador se detiene inmediatamente con un error de "variable guppy is already defined". Visto lo anterior, tenemos que el inciso correcto es el F. Pienso que es buena idea que con los pattern matching debemos prestar mucha atención a cuestiones como el operador de negación o el reintentar declarar una misma variable dentro de la zona donde cobró vida, ya que esto hará que el código no compile.

Pregunta número 29

29. What is the result of the following code?

- ```
1: public class PrintIntegers {
2: public static void main(String[] args) {
3: int y = -2;
4: do System.out.print(++y + " ");
5: while(y <= 5);
6: } }
```
- A.** -2 -1 0 1 2 3 4 5  
**B.** -2 -1 0 1 2 3 4  
**C.** -1 0 1 2 3 4 5 6  
**D.** -1 0 1 2 3 4 5  
**E.** The code will not compile because of line 5.  
**F.** The code contains an infinite loop and does not terminate.

**Answer 29:** En esta pregunta tenemos dos conceptos principales: por un lado, la omisión de llaves en los bucles y el operador de preincremento. Vamos a checar el código paso a paso para ver exactamente qué imprime la consola. Empecemos con la sintaxis del bucle que tenemos en las líneas 4 y 5. Debemos recordar que en Java, al igual que los if o los while, el bucle do-while puede omitir las llaves { } si su cuerpo consta de una única instrucción. Como la línea 4 solo tiene el

System.out.print(), esto es 100% legal y compila sin problemas. El estado inicial en la línea 3 nos da que  $y = -2$ . De ahí nos vamos a la ejecución del bucle: En la iteración 1 entra al do. La instrucción es  $++y$ . Como es preincremento, tenemos que la variable 'y' sube primero a -1 y luego se imprime. Lo que imprime: -1 . La condición de  $-1 \leq 5$  es true. De ahí vamos a las iteraciones 2 a 7: La variable sigue subiendo e imprimiéndose (0, 1, 2, 3, 4). Todo normal hasta este momento. En la iteración donde imprime 5, entra al do. y sube de 4 a 5. Imprime: 5 . Llega al while y evaluamos que la condición  $5 \leq 5$  es verdadera. Por lo tanto, el bucle da una vuelta más. El problema llega en la vuelta final. Entra al do de nuevo. Ejecuta  $++y$ , por lo que y sube a 6. Se imprime: 6 . Ahora se evalúa la condición  $6 \leq 5$ ; es falso. El bucle termina. El resultado final en consola es la secuencia -1 0 1 2 3 4 5 6. Con lo anterior dicho, tenemos que la respuesta correcta es el inciso C. Unos puntos que podemos resaltar son, primero, que debemos recordar que 'if', for, while y 'do-while' pueden omitir las llaves si solo ejecutan una línea. En el caso del do-while, la instrucción simple va entre el do y el while(condición). Lo segundo es que cuando veamos un menor o igual ( $\leq$ ), debemos tener cuidado en la iteración límite. Si la variable cumple exactamente el número de la condición, siempre dará una vuelta adicional donde el número inevitablemente se pasará del límite.